
O que é RegExp?

RegExp significa:

| **Regular Expression** (Expressão Regular)

Uma expressão regular é um padrão usado para procurar, validar, substituir ou manipular textos.

Ela funciona como um mecanismo de busca inteligente.

Para que RegExp é usado?

Regular Expressions são usadas em:

- Validação de formulários
- Verificação de emails
- Validação de senhas
- Busca de palavras
- Filtros de pesquisa
- Sistemas de login
- APIs
- Back-end
- Extração de dados
- Web Scraping
- Compiladores
- Editores de texto
- Processamento de arquivos
- Chatbots
- Sistemas de autenticação

Estrutura Básica

/padrao/flags

Exemplo

```
/gabriel/i
```

Parte	Significado
/	delimitador da expressão
gabriel	padrão de busca
i	flag/modificador

Criando RegExp

1. Forma Literal (mais comum)

```
const regex = /javascript/;
```

2. Construtor RegExp

```
const regex = new RegExp("javascript");
```

Muito usado quando o padrão vem de variáveis.

Exemplo:

```
const palavra = "javascript";  
  
const regex = new RegExp(palavra);
```

Métodos Principais

String.search()

Procura a posição da primeira ocorrência.

Exemplo

```
const texto = "Visit W3Schools";

const resultado = texto.search(/W3Schools/);

console.log(resultado);
```

Resultado

```
6
```

Porque `W3Schools` começa na posição 6.

String.match()

Retorna o que foi encontrado.

Exemplo

```
const texto = "red green blue";

console.log(texto.match(/red|green|blue/g));
```

Resultado

```
["red", "green", "blue"]
```

String.replace()

Substitui texto.

Exemplo

```
const texto = "Visit Microsoft";

const resultado = texto.replace(/Microsoft/, "Google");
```

```
console.log(resultado);
```

Resultado

Visit Google

RegExp.test()

Verifica se o padrão existe.

Retorna:

- true
- false

Exemplo

```
const regex = /Gabriel/;  
  
console.log(regex.test("Gabriel Silva"));
```

Resultado

true

Flags

Flags modificam o comportamento da RegExp.

Estrutura

```
/padrao/flags
```

Flag g

Global

Procura TODAS as ocorrências.

Sem g

```
"banana".match(/a/)
```

Resultado:

```
["a"]
```

Apenas o primeiro.

Com g

```
"banana".match(/a/g)
```

Resultado:

```
["a", "a", "a"]
```

Flag i

Case Insensitive

Ignora letras maiúsculas e minúsculas.

```
/gabriel/i
```

Aceita:

- Gabriel
- GABRIEL
- gabriel

Flag u

Unicode

Permite suporte Unicode.

Muito importante para emojis e caracteres especiais.

```
const texto = "😊";  
  
console.log(/😊/u.test(texto));
```

Metacaracteres

Metacaracteres são caracteres especiais que possuem significado próprio.

\d

Digits

Procura números.

```
/\d/
```

Exemplo

```
const texto = "Casa 123";  
  
console.log(texto.match(/\d/g));
```

Resultado:

```
["1", "2", "3"]
```

\w

Word Characters

Procura:

- letras
- números
- underscore `_`

```
/\w/
```

Exemplo

```
const texto = "Gabriel_123";  
  
console.log(texto.match(/\w/g));
```

\s

Spaces

Procura espaços.

```
/\s/
```

Exemplo

```
const texto = "Olá Mundo";  
  
console.log(texto.match(/\s/g));
```

Resultado:

```
[ " " ]
```

. (ponto)

Representa qualquer caractere.

```
/. /
```

Exemplo:

```
/g.t/
```

Aceita:

- gat
- get
- git
- g9t

Character Classes

Usam colchetes `[]`.

Permitem definir conjuntos de caracteres.

[abc]

Aceita:

- a
- b
- c

```
/[abc]/
```

[a-z]

Aceita letras minúsculas.

```
/[a-z]/
```

[A-Z]

Aceita letras maiúsculas.

```
/[A-Z]/
```

[0-9]

Aceita números.

```
/[0-9]/
```

abc

O `^` dentro dos colchetes significa NEGAÇÃO.

```
/[^abc]/
```

Aceita qualquer caractere EXCETO:

- a
 - b
 - c
-

Quantifiers

Quantificadores definem quantas vezes algo deve aparecer.

Zero ou mais

```
/10*/
```

Aceita:

- 1
 - 10
 - 100
 - 1000
-

+

Um ou mais

```
/10+/
```

Aceita:

- 10
- 100
- 1000

Não aceita:

- 1
-

?

Zero ou um

```
/10?/
```

Aceita:

- 1
 - 10
-

{n}

Quantidade exata

```
/\d{3}/
```

Exatamente 3 números.

{n,m}

Intervalo

```
/\d{2,5}/
```

Aceita:

- 2 números
 - 3 números
 - 4 números
 - 5 números
-

Assertions

Assertions verificam posições.

Início da string

```
/^Gabriel/
```

Aceita:

```
"Gabriel Silva"
```

Rejeita:

```
"Olá Gabriel"
```

\$

Final da string

```
/Silva$/
```

Aceita:

```
"Gabriel Silva"
```

Rejeita:

```
"Silva Gabriel"
```

\b

Word Boundary

Limite de palavra.

```
/\bcat\b/
```

Aceita:

```
cat
```

Não aceita:

```
catalog
```

Alternation |

Significa OU.

```
/red|green|blue/
```

Procura:

- red
- green
- blue

Lookaheads

Verificam o que vem DEPOIS.

Exemplo

```
/\d(=?px)/
```

Aceita números apenas se forem seguidos de px .

Exemplo

```
const texto = "10px";  
  
console.log(texto.match(/\d(=?px)/g));
```

Resultado:

```
[ "0" ]
```

Lookbehind

Verifica o que vem ANTES.

```
/(?<=R\$)\d+/
```

Exemplo

```
const texto = "R$500";  
console.log(texto.match(/(?<=R\$)\d+/));
```

Resultado:

```
[ "500" ]
```

Escapando Caracteres

Alguns caracteres possuem significado especial.

Para procurar literalmente usamos `\`.

Exemplo

```
/\./
```

Procura um ponto literal.

Sem escape:

```
./.
```

Significa qualquer caractere.

Exemplo de Escape

```
const texto = "www.site.com";  
  
console.log(texto.match(/\.\/g));
```

Resultado:

```
[".", "."]
```

Métodos Importantes Extras

exec()

Executa busca detalhada.

```
const regex = /abc/;  
  
console.log(regex.exec("123 abc 456"));
```

split()

Divide texto usando regex.

```
const texto = "banana,maçã,uva";  
  
console.log(texto.split(/,/));
```

Resultado:

```
[ "banana", "maçã", "uva" ]
```

Exemplos Reais

Validar Email

```
const email = "teste@gmail.com";

const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

console.log(regex.test(email));
```

Explicação

Parte	Significado
^	começo
[^\s@]+	caracteres antes do @
@	arroba
\.	ponto
\$	fim

Validar Telefone

```
const telefone = "819999999999";

const regex = /^\d{11}$/;

console.log(regex.test(telefone));
```

Validar CPF Simples

```
const cpf = "12345678900";

const regex = /^\\d{11}$/;

console.log(regex.test(cpf));
```

Validar Senha Forte

```
const senha = "abc123";

const regex = /^(?=.*[A-Za-z])(?=.*\\d){6,}$/;

console.log(regex.test(senha));
```

Explicação

Parte	Significado
<code>(?=.*[A-Za-z])</code>	precisa ter letras
<code>(?=.*\\d)</code>	precisa ter números
<code>{6,}</code>	mínimo 6 caracteres

Capturing Groups

Usam parênteses `()`.

Permitem capturar partes do texto.

Exemplo

```
const texto = "2026-05-13";

const regex = /(\\d{4})-(\\d{2})-(\\d{2})/;
```

```
console.log(texto.match(regex));
```

Resultado:

```
[  
  "2026-05-13",  
  "2026",  
  "05",  
  "13"  
]
```

Explicação

Grupo	Valor
Grupo 1	ano
Grupo 2	mês
Grupo 3	dia

Substituição com Groups

```
const data = "2026-05-13";  
  
const resultado = data.replace(  
  /(\d{4})-(\d{2})-(\d{2})/,  
  "$3/$2/$1"  
);  
  
console.log(resultado);
```

Resultado:

```
13/05/2026
```

Regex em Loops

```
const texto = "1 2 3 4 5";

const numeros = texto.match(/\d/g);

for (let numero of numeros) {
  console.log(numero);
}
```

Cuidados Importantes

Regex pode ficar difícil

Evite criar expressões gigantes sem comentários.

Regex não substitui lógica

Nem tudo deve ser feito com RegExp.

Regex muito pesada pode impactar performance

Especialmente em:

- arquivos grandes
- loops gigantes
- validações em massa

Dicas Profissionais

- Comece simples
- Teste passo a passo
- Use sites de teste Regex
- Comente expressões complexas

- Use nomes claros
-

Ferramentas úteis

Regex101

Excelente para testar regex.

RegExr

Ferramenta visual muito boa.

Resumo Final

Conceito	Função
/texto/	padrão
g	global
i	ignore case
\d	números
\w	letras/números
\s	espaços
[]	conjunto
()	grupos
*	zero ou mais
+	um ou mais
?	opcional
^	começo
\$	fim
,	,

Conclusão

Regular Expressions são uma das ferramentas mais poderosas do JavaScript.

Dominar RegExp permite:

- validar dados
- criar filtros avançados
- manipular texto
- construir sistemas profissionais
- trabalhar melhor com APIs
- desenvolver back-end
- automatizar tarefas

É um dos conhecimentos mais importantes para qualquer desenvolvedor JavaScript profissional.